SWIFTDROP · DESIGN DOCUMENTATION · PASIDU MIHIRANGA

Reporting and AI Assistant

Design of Pasidu Mihiranga's two services: event-driven delivery reports for dispatchers and admins, and a delivery assistant that answers customers by calling SwiftDrop's own APIs, also available as MCP tools.

Services reporting-service (Spring Boot 4) · ai-assistant-service (FastAPI) **Data** PostgreSQL reporting_db · Redis · Kafka **Issue** #23

1 Use cases

2 Data model

3 Architecture

4 Integration design

5 Key flows

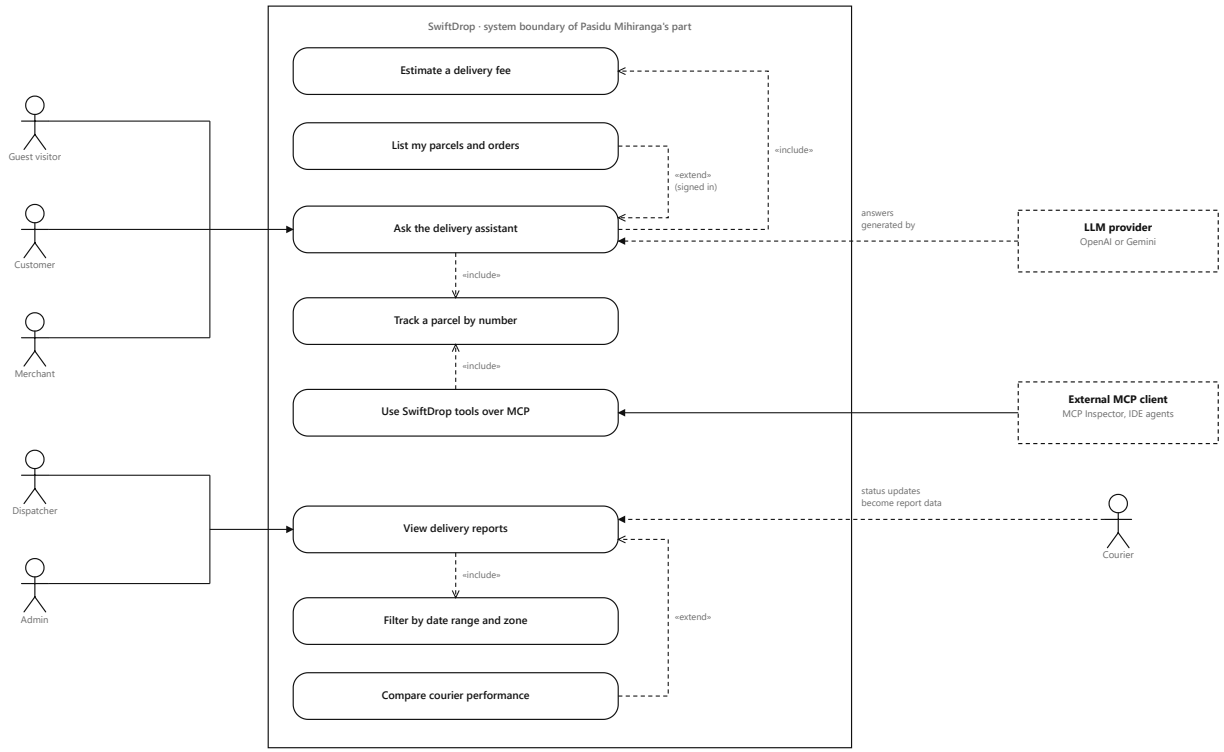
6 Design patterns and principles

Diagram sources: `doc/diagrams/m10/*.svg` in the SwiftDrop repository, with matching PNG exports for the design PDF. Example values (fees, tracking numbers) are illustrative.

1 Use cases

Pasidu Mihiranga owns two capabilities. **Reporting** gives dispatchers and admins delivery volumes and success rates. The **delivery assistant** answers questions such as *"Where is my parcel?"* and *"How much to send 2 kg to Kandy?"* for guests, customers and merchants, and exposes the same tools to external MCP clients.

FIGURE 1 **Use cases: reporting and the delivery assistant**

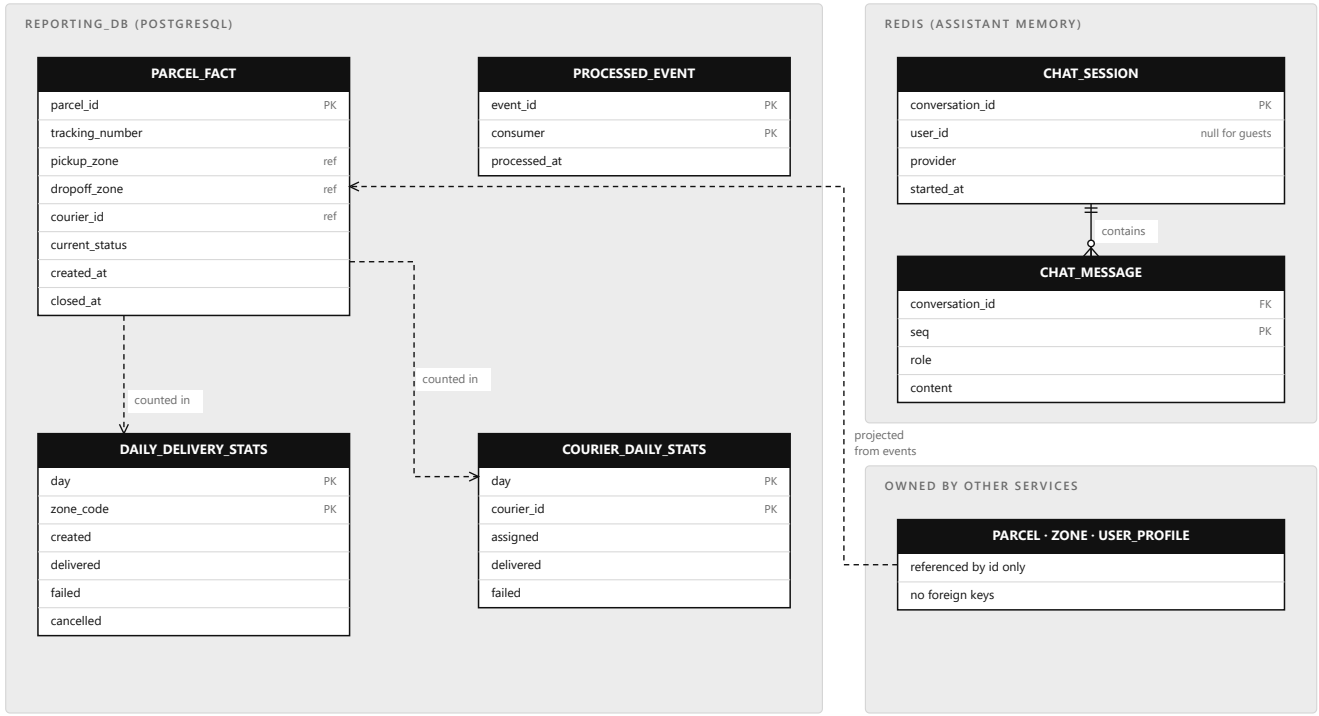


Customers, merchants and guests use the assistant; dispatchers and admins read reports. Couriers have no direct use case here: their status updates are the data the reports count. «include» points to the use case that is always included; «extend» points to the use case it extends.

2 Data model

Reports are **read models**: reporting-service never queries another service's database. It keeps a small `PARCEL_FACT` projection (zone and courier of every parcel) so a later status event can be counted against the right zone and courier, and aggregates into daily tables that the dashboard reads directly. `PROCESSED_EVENT` makes event handling idempotent.

FIGURE 2 **Data model: reporting projections and assistant memory**



reporting_db holds read models built from Kafka events. Chat sessions live in Redis with a 24-hour TTL. Entities owned by other services are referenced by id only, never by foreign key. Solid line with crow's foot: a real one-to-many relationship. Dashed arrow: rows derived by the projection, with no foreign key.

3 Architecture

Proposal: microservices behind an API gateway, with the Next.js app acting as backend-for-frontend (BFF). Each service owns its data; services integrate through Kafka events or through the gateway.

Why this style for Pasidu Mihiranga's services

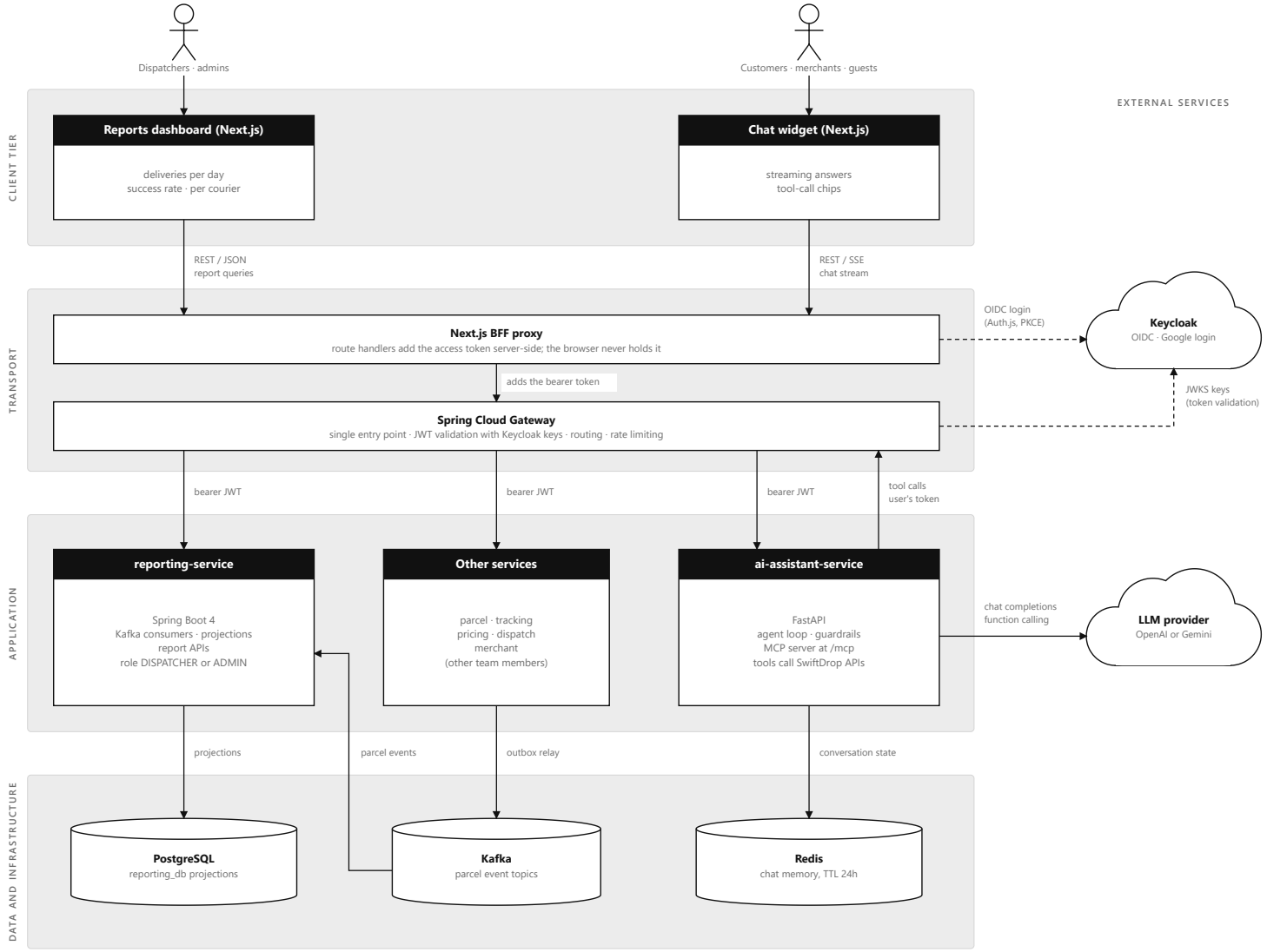
- **Independent scaling:** report reads and LLM-bound chat have very different load profiles from parcel writes.
- **Different runtimes:** Java suits transactional projections; Python has the strongest LLM and MCP libraries. Microservices let each use the right one.
- **Loose coupling:** reporting subscribes to events, so producers never know it exists and a reporting outage never blocks bookings or deliveries.
- **Security by construction:** the assistant has no database access of its own; it can only do what the user's token allows through the gateway.

Tiers

Client	Next.js chat widget and reports dashboard
Transport	Next.js BFF and Spring Cloud Gateway: one entry point, JWT validation
Application	reporting-service (api → application → domain → infrastructure), ai-assistant-service (api → agent → tools)
Data	PostgreSQL `reporting_db`, Redis chat memory, Kafka topics

Trade-off accepted: reports are eventually consistent (typically under a second behind).

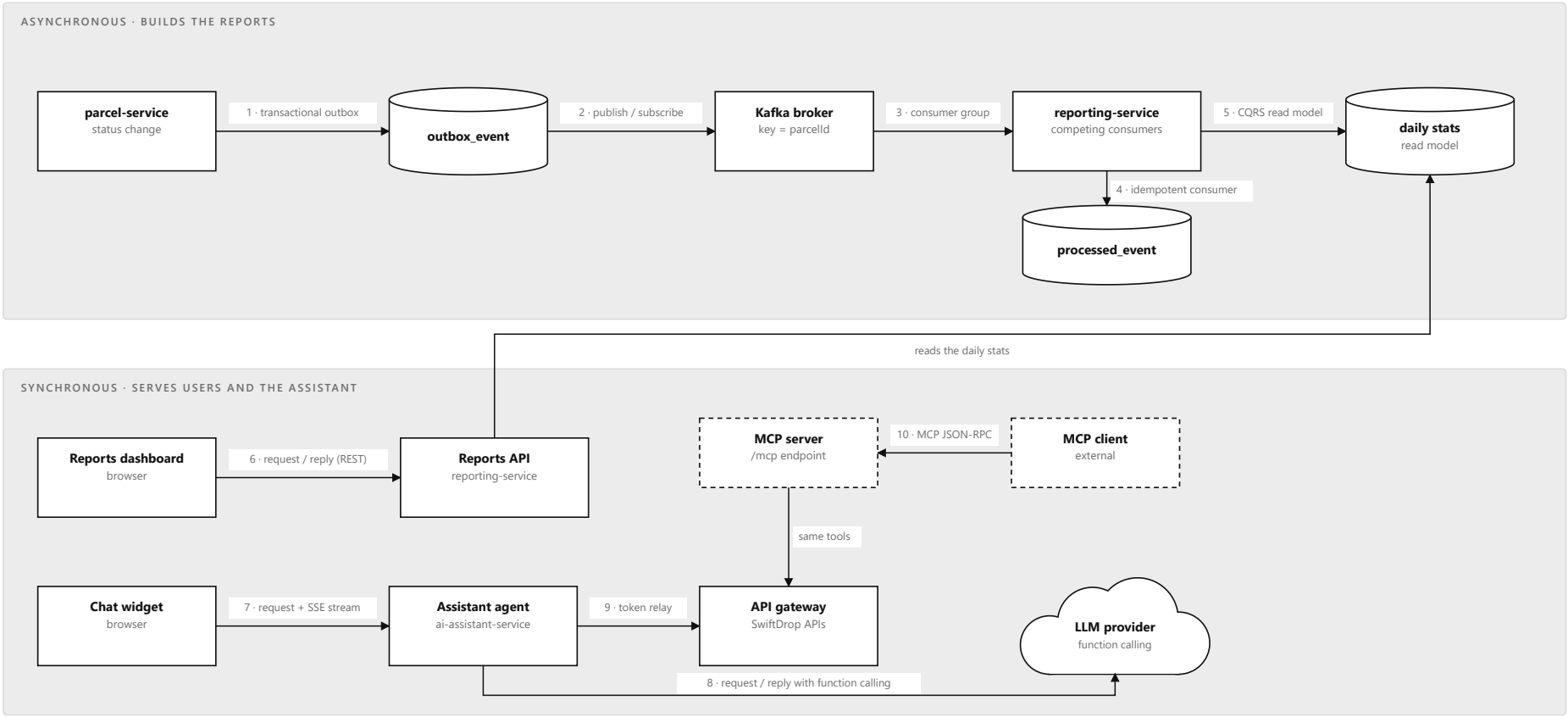
FIGURE 3 **Architecture: layered microservices with a BFF**



Four tiers. Pasidu Mihiranga's services sit in the application tier; reports are read models fed by Kafka, and the assistant reaches every other service only through the gateway with the user's token.

4 Integration design

FIGURE 4 **Integration design: the patterns Pasidu Mihiranga's services rely on**



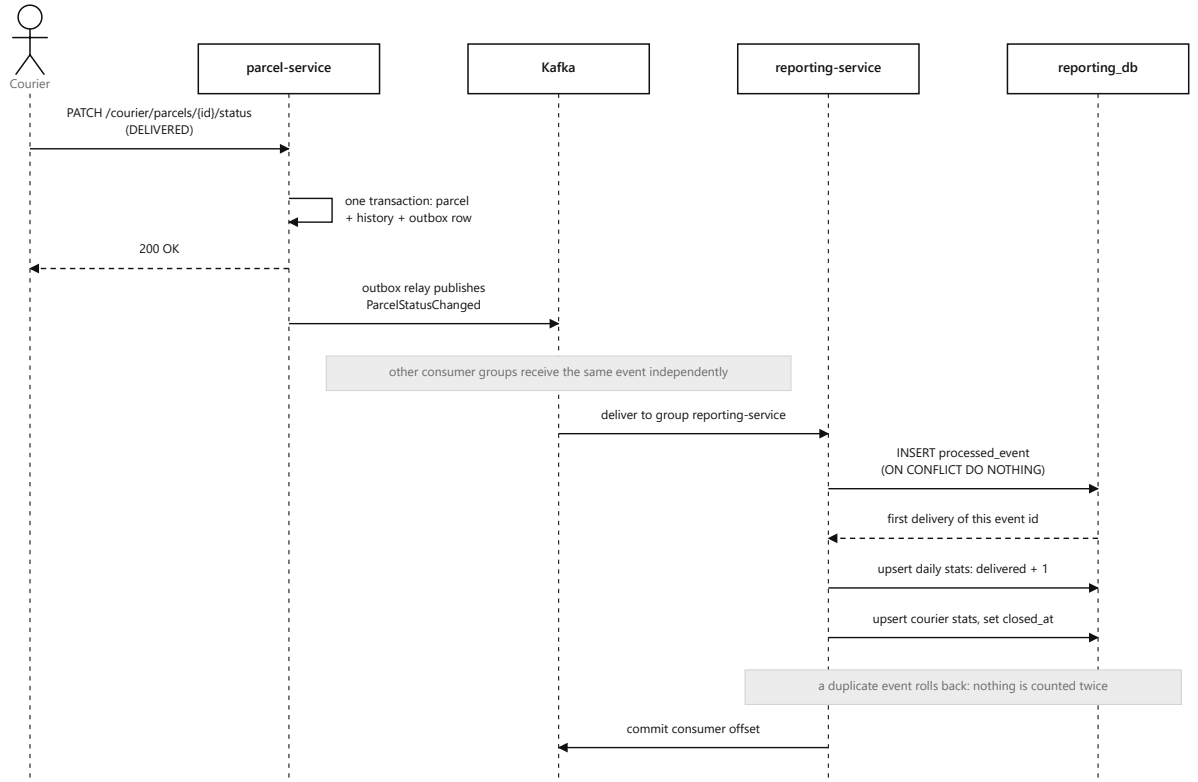
Numbers match the pattern table. Asynchronous patterns (1-5) build the reports; synchronous ones (6-10) serve users and the assistant.

#	PATTERN	WHERE USED	WHY
1	Transactional outbox	parcel-service writes the event with the status change	Event and data commit together, so reports never miss or invent a delivery.
2	Publish/subscribe (broker)	Kafka topics `swiftdrop.parcel.*.v1`	One status change feeds reports, notifications, webhooks and tracking without the producer knowing any of them.
3	Competing consumers	Consumer group `reporting-service`	Replicas share partitions; key = parcelId keeps each parcel's events in order.
4	Idempotent consumer	`processed_event` table	Kafka delivers at least once; duplicates must not inflate the numbers.
5	CQRS read model	Daily and courier stats tables	Dashboards read pre-aggregated rows instead of joining across services.
6	Request/reply (REST)	`/api/v1/reports/**` via gateway	The dashboard needs an immediate answer for a chosen date range.
7	Streaming (Server-Sent Events)	`POST /api/v1/assistant/chat`	Users see the answer and tool activity as it is produced.
8	Request/reply with function calling	Assistant to OpenAI or Gemini	The model decides which SwiftDrop tool to call; answers stay grounded in real data.
9	Gateway + token relay	Tool calls carry the user's JWT	The assistant gets exactly the user's permissions, never more.
10	MCP (JSON-RPC over HTTP)	`/mcp` server	Standard protocol so external AI clients can reuse the same tools.

5 Key flows

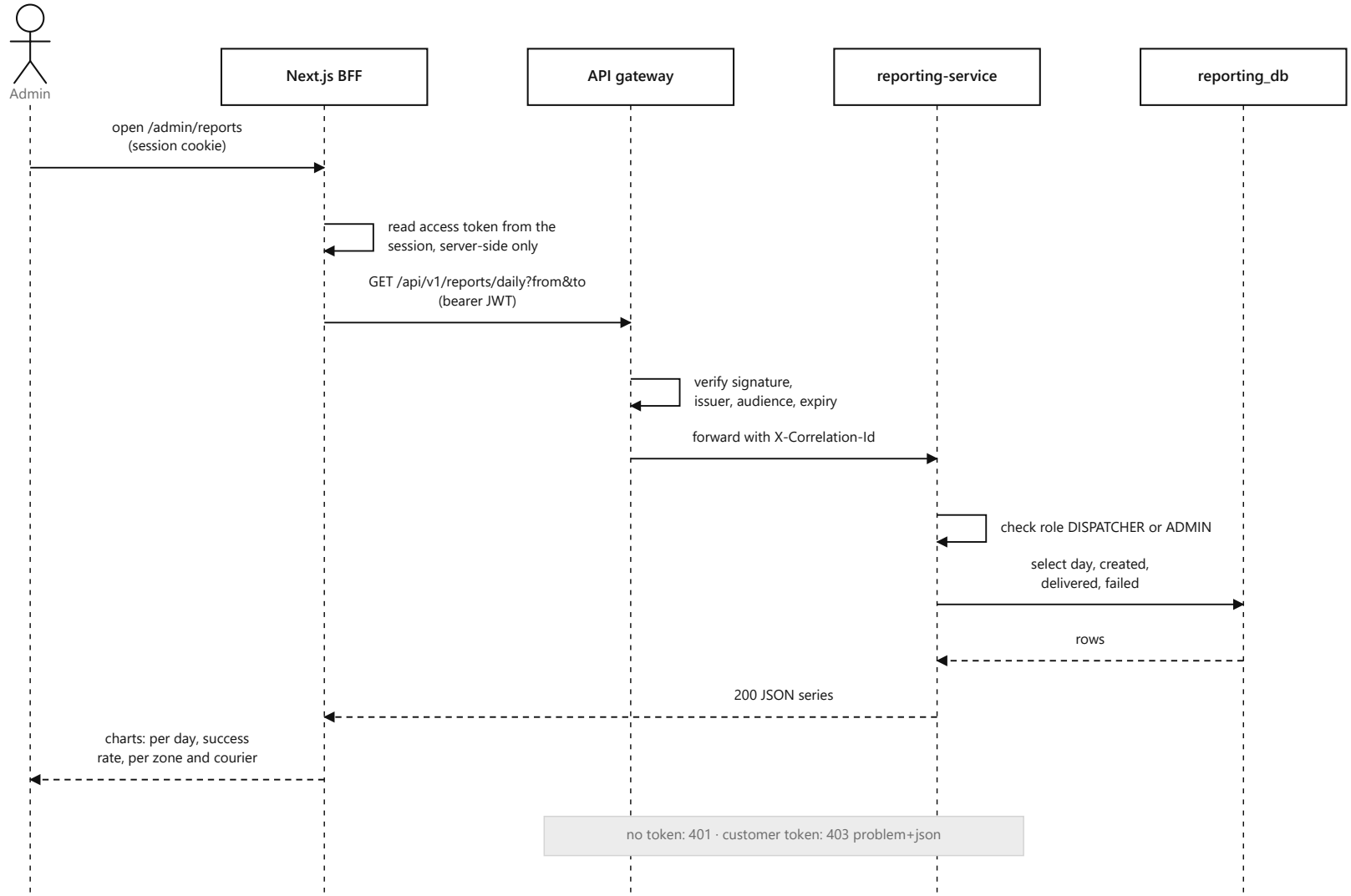
Flow 5 is Pasidu Mihiranga's part of the shared *courier status update* flow. Flows 7 to 10 cover the assistant, including how a user's login (OAuth 2.0 / OIDC through Keycloak) carries through to every tool call.

FIGURE 5 **Sequence: a courier status update becomes report data**



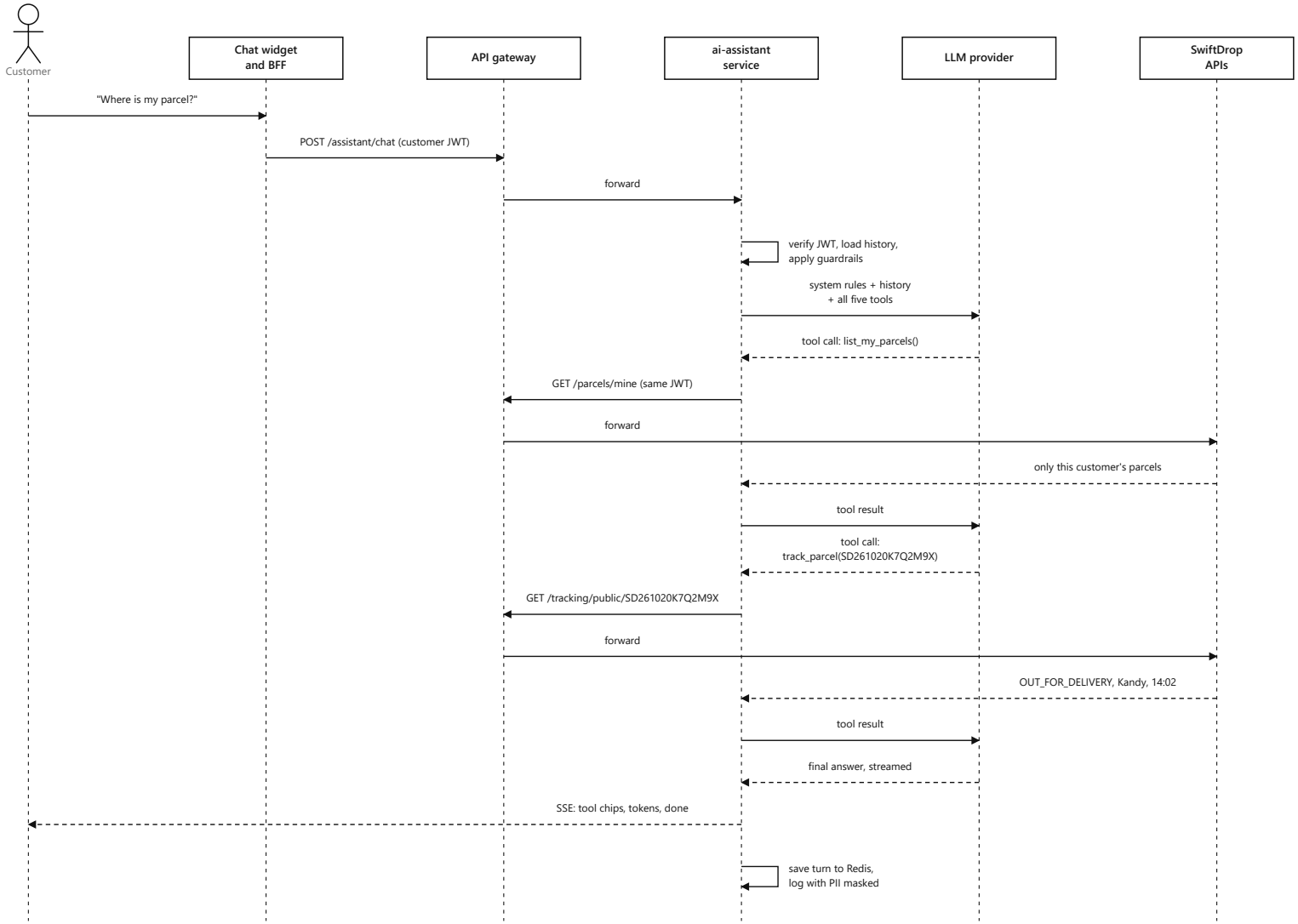
The event that triggers notifications and webhooks is consumed independently by reporting-service. A duplicate delivery is detected and counted only once.

FIGURE 6 **Sequence: an admin opens the reports dashboard**



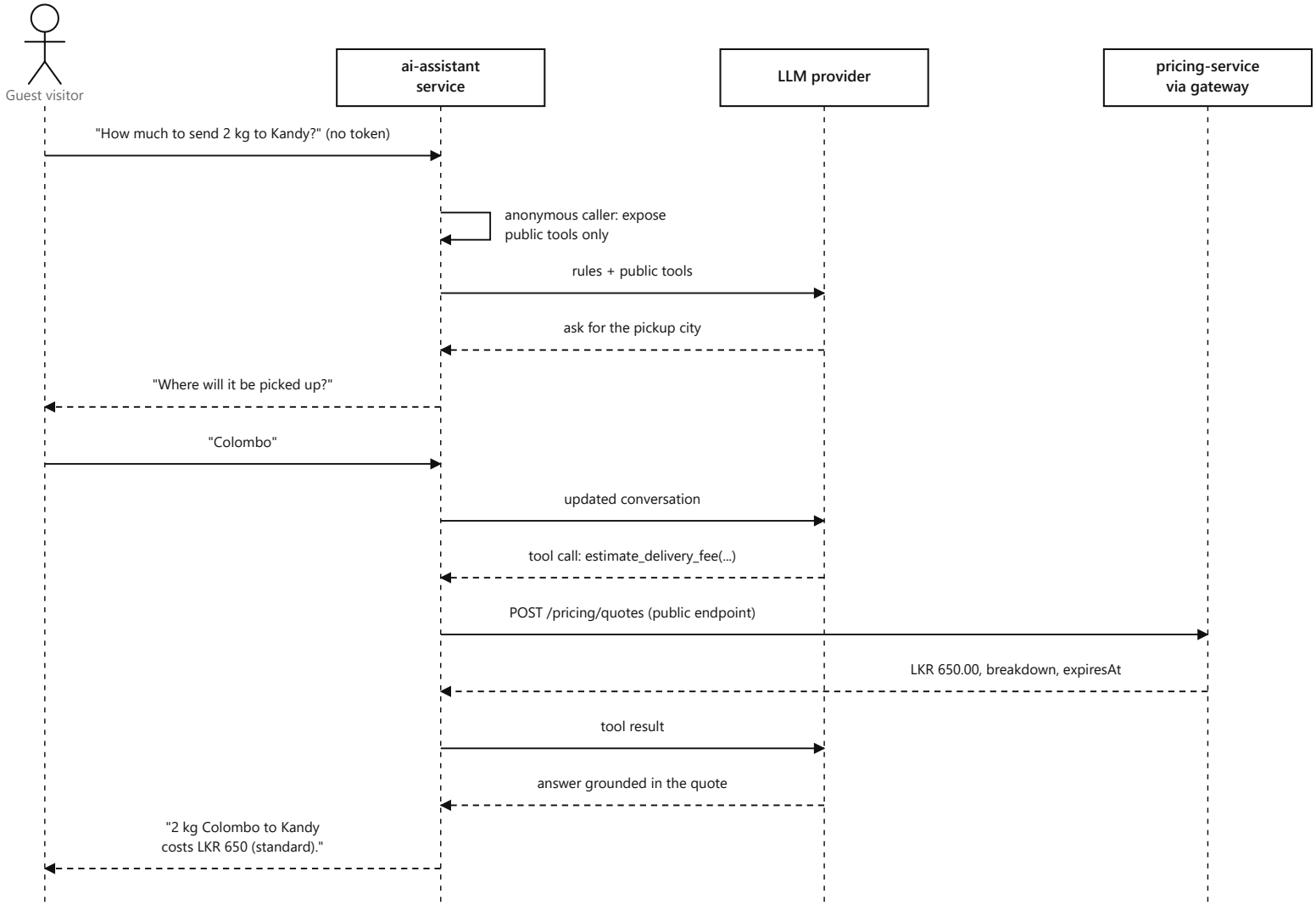
The browser never holds the access token: the BFF adds it server-side. Gateway and service both validate it, and the service enforces the DISPATCHER or ADMIN role.

FIGURE 7 **Sequence: "Where is my parcel?" for a signed-in customer**



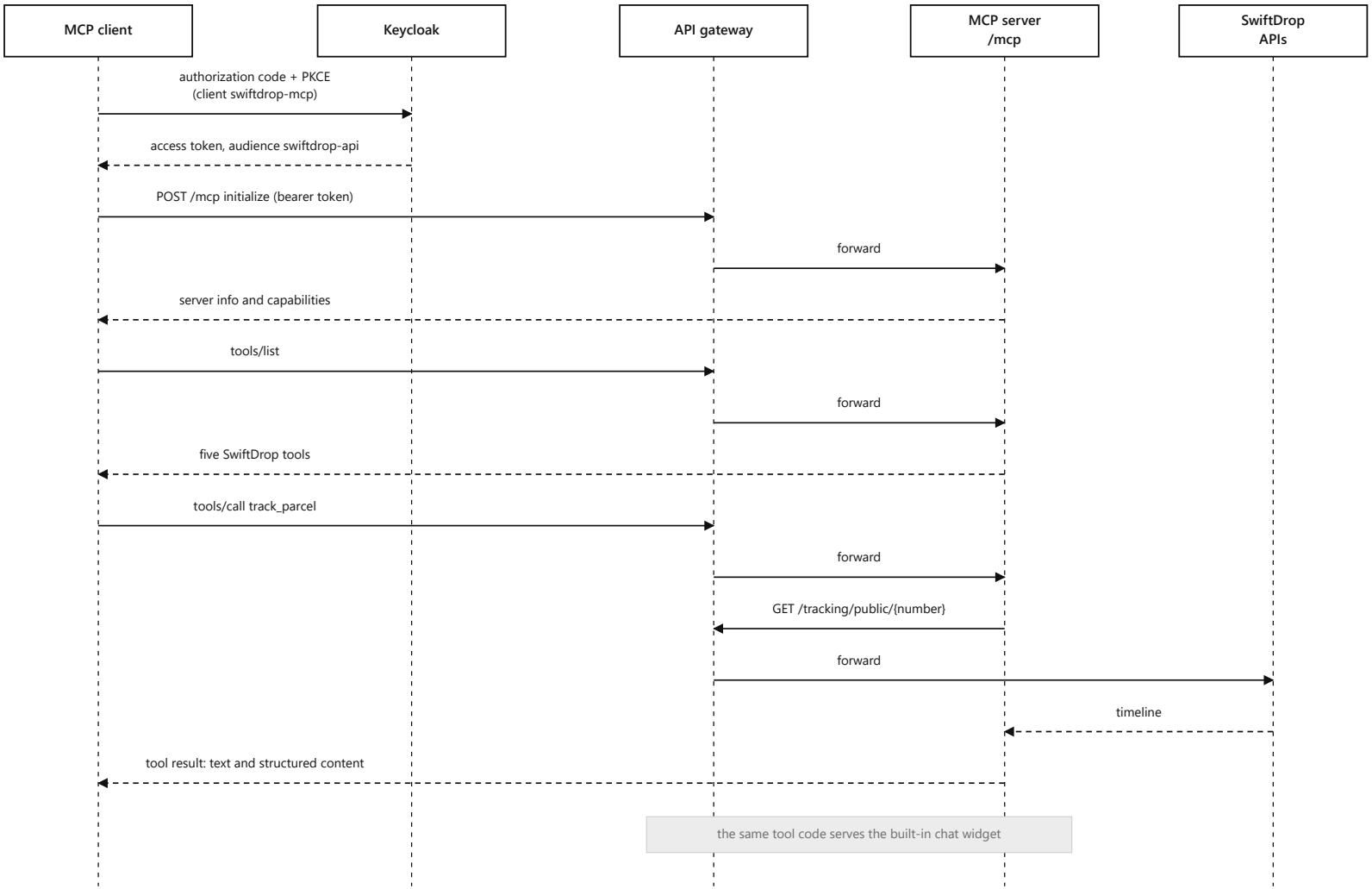
The assistant calls the same APIs as the web app with the customer's own token, so it can only see that customer's parcels. The answer streams back over SSE.

FIGURE 8 **Sequence: a guest asks for a delivery fee**



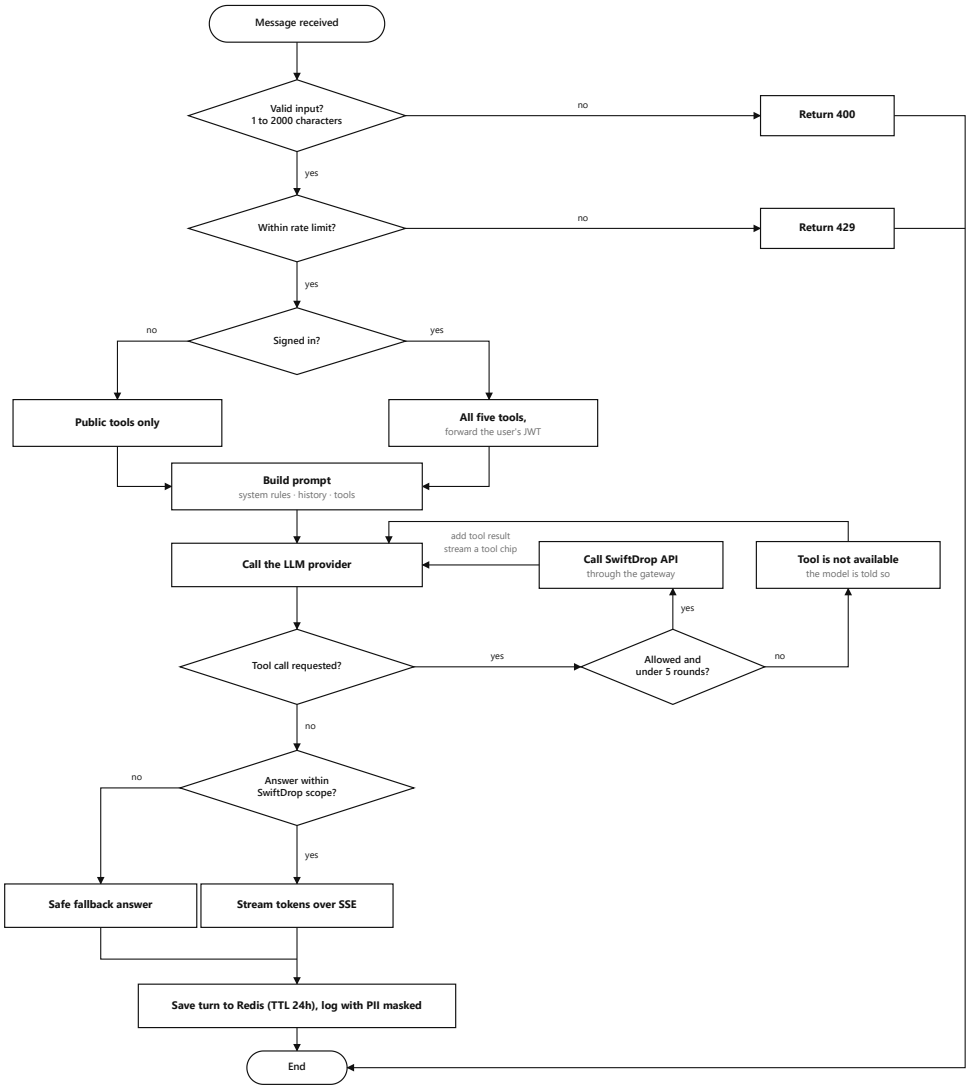
Guests get public tools only. With the pickup city missing, the model asks for it before calling the pricing tool. The fee shown is an example value.

FIGURE 9 **Sequence: an external MCP client uses SwiftDrop tools**



The same tool implementations serve the built-in chat and any MCP client. The client signs in through Keycloak and the token travels with every tool call.

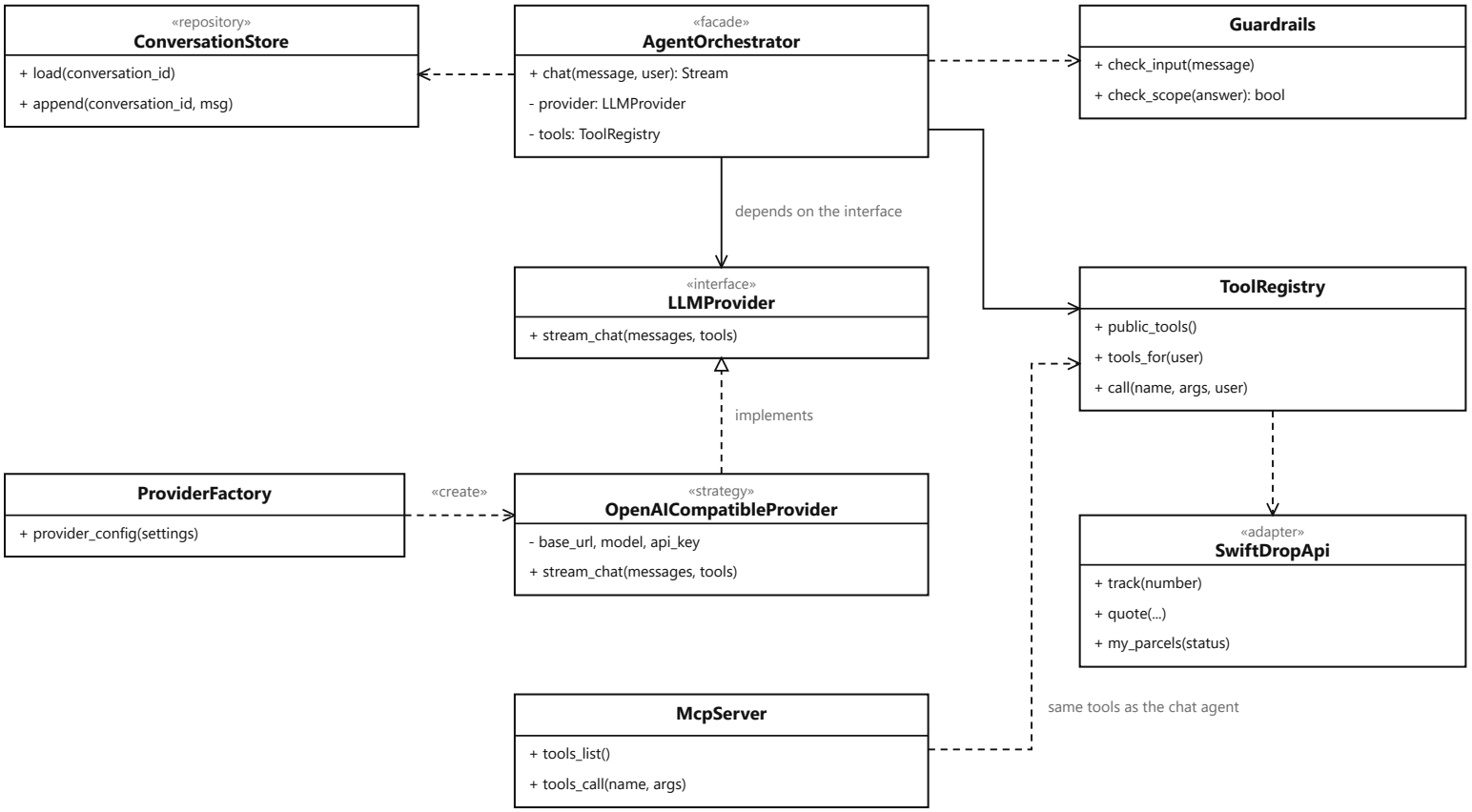
FIGURE 10 **Activity: how the assistant answers one message**



Every guardrail is a decision in this loop: input limits, rate limiting, tool scope by sign-in state, an iteration cap, and a scope check before anything is streamed back.

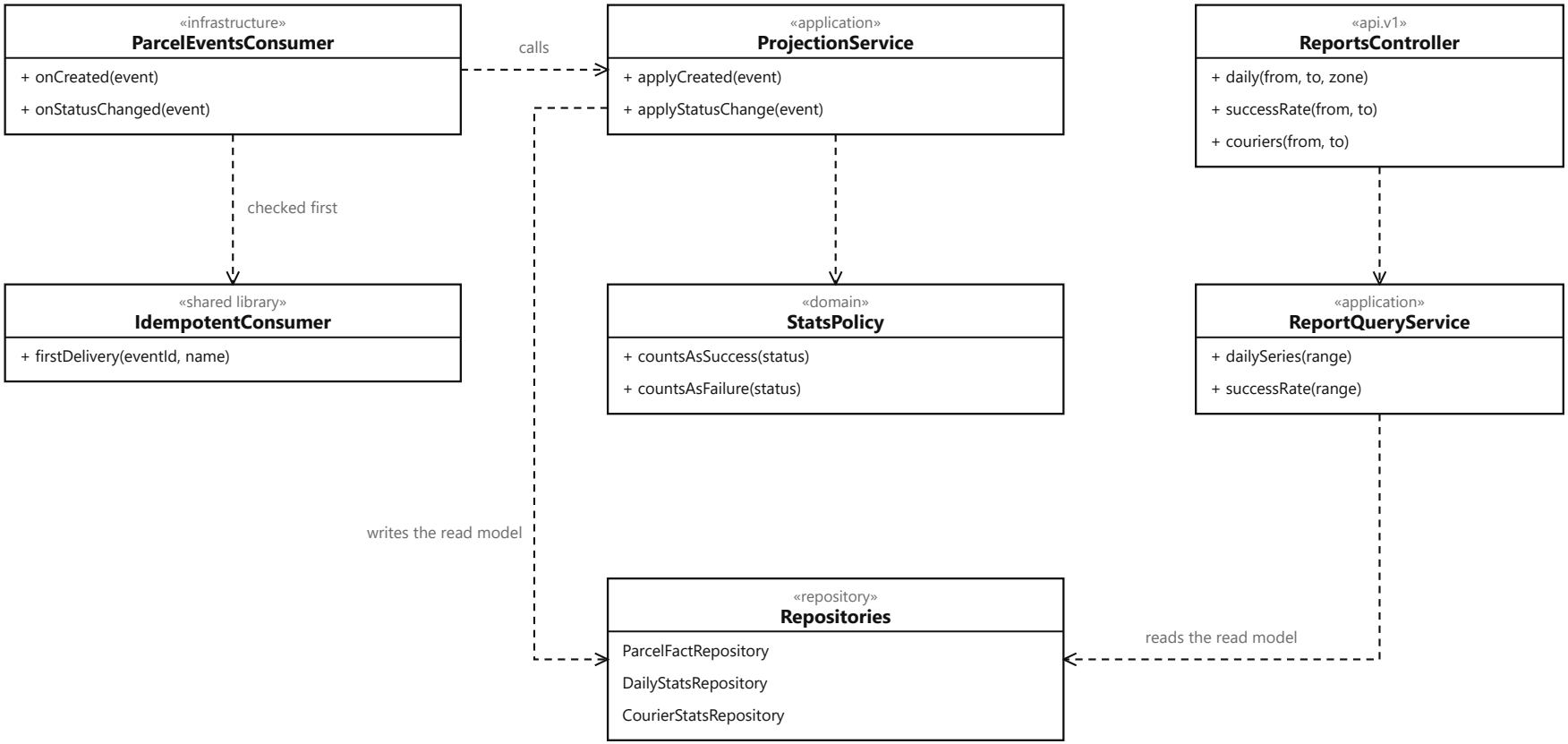
6 Design patterns and principles

FIGURE 11 **Design patterns in the AI assistant**



Strategy for the LLM provider, a factory that picks it from configuration, an adapter over the gateway, and one tool registry shared by the chat agent and the MCP server. Dashed open arrow: depends on. Solid open arrow: holds a field of that type. Dashed hollow triangle: implements.

FIGURE 12 **Design patterns in reporting-service**



Layered and hexagonal: consumer and controller are adapters around application services, repositories hide persistence, and the idempotent consumer comes from the shared library. Dashed open arrows are dependencies; the domain policy depends on nothing.

PATTERN	APPLIED TO
Strategy	`LLMProvider`: OpenAI and Gemini are interchangeable; `LLM_PROVIDER` selects one at start-up.
Factory	`provider_config(settings)` builds the provider from environment variables.
Adapter	`SwiftDropApi` wraps gateway HTTP calls as typed tool functions; Kafka consumer and REST controller adapt the outside world to application services.
Facade	`AgentOrchestrator` hides the model loop, tools, memory and guardrails behind one `chat()` call.
Observer / pub-sub	reporting-service reacts to parcel events it subscribes to.
Repository	Spring Data repositories for stats; `ConversationStore` for Redis chat memory.
CQRS and idempotent consumer	Write side lives in parcel-service; reporting keeps its own read model and skips duplicates.
Template method	Shared consumer pipeline: dedupe, handle, commit (libs/java and libs/node).

Principles

- **Single responsibility:** consumers only translate events; projection rules live in `StatsPolicy`; controllers only validate and delegate.
- **Open/closed:** a new LLM provider or a new MCP tool is an added class, not an edit to the agent.
- **Dependency inversion:** the agent depends on `LLMProvider` and `ToolRegistry` interfaces; ArchUnit enforces the layer rules in reporting-service.
- **DRY:** one tool registry serves both the chat and the MCP server; JWT, logging and error handling come from the shared libraries.
- **Least privilege:** token relay means the assistant inherits the caller's rights; guests get public tools only.
- **Twelve-factor configuration:** provider, model and keys come from environment variables; no secret is stored in code.